SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 - Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 - Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills		
Student Name:	**Kosicram K**	**Reg. No.:**	**192571079**
Date:	**26/10/26**	**Duration:**	60 minutes

Code of Conduct

I Kosicram K / 192571079 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **Kosicram K**

Annexure A - Architecture Design Assignment

Based on your assigned problem statement, perform an Architecture Design Analysis and prepare an architecture design report by answering the following questions:

> **IntelliAssess – An AI-Driven Online Testing, Automated Evaluation and Examination Monitoring Platform**

Problem Statement: Traditional examination systems require significant manual effort for test administration, answer evaluation, and result processing, making them time-consuming and prone to errors. Existing online examination platforms often lack integrated intelligent evaluation and effective monitoring of suspicious activities. IntelliAssess aims to provide an AI-driven platform that automates assessment, accelerates result generation, and monitors examinations

1. Analyze System Requirements

1.1 Study the problem statement and identify the system objectives

IntelliAssess must conduct large, time-bound online tests fairly and reliably while replacing manual administration, evaluation and result processing with AI. It is a bursty, integrity-critical, multi-role platform: load is near zero most of the day but spikes sharply when all candidates log in and again when they submit, and AI scoring and monitoring add heavy compute that must never slow the candidate; a lost answer, wrong score or undetected malpractice has serious academic and legal consequences.

1. **Secure test management:** create questions (with AI-assisted generation and tagging), build papers (random / sectioned / negative marking), schedule tests and keep papers encrypted until the start time.
2. **Reliable test delivery:** authenticate candidates, deliver the paper with a server-authoritative timer, auto-save answers continuously and accept submission even after temporary network loss.
3. **AI-based exam monitoring:** detect and record suspicious behaviour (tab switch, multiple faces, face absent, gaze deviation, copy-paste, second screen) and give proctors a live risk dashboard.
4. **Automated evaluation:** auto-grade objective questions in seconds; score descriptive answers with an NLP / LLM engine against rubrics and model answers; route low-confidence answers to examiners; publish results after approval.
5. **Integration:** single sign-on with the institution directory and two-way data exchange with the ERP / SIS / LMS (students, courses, marks).
6. **Analytics and compliance:** item analysis, pass-percentage dashboards, tamper-evident audit trail for accreditation (NAAC / NBA) and re-evaluation disputes.

7. **Extensibility:** add new question types, AI evaluation or proctoring models and institutions without rewriting the platform.

1.2 Functional requirements that influence the architecture

ID	Functional Requirement	Architectural Influence
FR1	Manage users, roles (student, faculty, exam cell, proctor, admin, auditor) and SSO login	Dedicated Identity & Access service, RBAC, token-based (JWT / OIDC) security at the gateway
FR2	Question bank with AI-assisted question generation, paper setting, scheduling and encrypted release at start time	Test Management service with AI question-assist module and its own database; time-locked, encrypted paper store
FR3	Deliver timed exam; server-side timer; auto-save and resume; final submit	Stateless Exam Delivery service + in-memory cache + client-side local buffer; idempotent save API
FR4	Capture webcam snapshots, browser events and heartbeats; compute AI risk score; alert proctors live	Persistent WebSocket channel, event streaming and asynchronous AI analysis workers
FR5	Auto-grade objective answers; AI scoring of descriptive answers against rubrics; confidence-based examiner review; moderation	Event-driven grading workers, AI Evaluation engine (NLP / LLM), separate Evaluation service, approval workflow
FR6	Publish results, mark sheets (PDF) and push marks to the ERP	Report generator worker, ERP integration adapter (anti-corruption layer)
FR7	Notifications (hall ticket, reminders, result published) by e-mail / SMS / in-app	Notification service consuming events; external gateways
FR8	Dashboards, item analysis, AI-monitoring reports, audit trail and re-evaluation records	Read-optimised analytics store; immutable audit service

1.3 Non-functional requirements that influence the architecture

Category	Measurable Requirement	Architectural Influence
Performance	Answer-save API p95 < 300 ms at 5,000 concurrent users; login burst of 5,000 users within 5 minutes; paper load < 3 s; AI scoring < 30 min	Redis caching, CDN for static assets, async writes, connection pooling, autoscaled AI workers
Scalability	10x growth (50,000 candidates) and multi-campus use without redesign; scale up only during exam windows	Stateless services, horizontal auto-scaling, partitioned event stream
Availability	99.9% during exam windows; no single point of failure; planned maintenance outside exams	Multi-AZ deployment, load balancer health checks, replicated DB and broker
Reliability / Durability	Zero loss of saved answers; RPO <= 5 minutes, RTO <= 30 minutes; resume after network or browser crash	Client local buffer with retry, write-ahead log, event replay, backups

Security	MFA for staff; TLS 1.3; encrypted papers and snapshots; RBAC; protection against impersonation, SQL injection, DDoS	WAF, API gateway, secrets vault, encryption at rest, least-privilege roles
Maintainability	New question type, AI model or proctoring rule released independently within one sprint; >= 80% unit-test coverage	Domain-based service boundaries, versioned APIs, CI/CD pipeline, contract tests
Auditability / Compliance	Every login, answer change, flag, AI score and mark change logged immutably; data retained per institution policy; privacy of webcam data	Append-only audit store, retention policies, role-restricted access to recordings
Usability / Interoperability	Works on low-end laptops and 2 Mbps links; REST / OIDC / SAML integration with ERP and SSO	Lightweight PWA, adaptive snapshot quality, standard protocols

1.4 Key quality attributes (ranked by influence on the architecture)

Rank	Quality Attribute	Why it dominates in IntelliAssess
1	**Availability & Reliability**	A test cannot be repeated casually; downtime or lost answers during a 2-hour window directly harms thousands of students.
2	**Scalability & Performance**	Extreme, predictable load spikes (login and submit) require elastic capacity and low latency.
3	**Security & Integrity**	Paper leakage, impersonation, cheating and unfair AI scoring destroy the credibility of the assessment.
4	**Maintainability & Extensibility**	Exam rules, question types and AI models change every semester; parts must evolve independently.
5	**Auditability**	Results are legal records and may be challenged; every AI score and human override must be provable.

2. Select a Suitable Software Architecture

2.1 Selected architectural style

Hybrid Architecture = Layered (N-tier) Client-Server + Domain-based Microservices + Event-Driven Processing. Clients (browser PWA) talk through an edge tier (CDN, WAF, API gateway) to a set of small, domain-aligned stateless microservices (Test Management, Test Delivery, AI Proctoring, AI Evaluation, Identity, ...). Time-consuming or bursty work (auto-grading, AI answer scoring, AI proctor analysis, report generation, notifications) is decoupled with an event broker (Apache Kafka) and handled by independent worker pools. Each service owns its data (database-per-service), and the whole system is deployed on containers (Kubernetes) across multiple availability zones.

2.2 Evaluation of candidate architectural styles

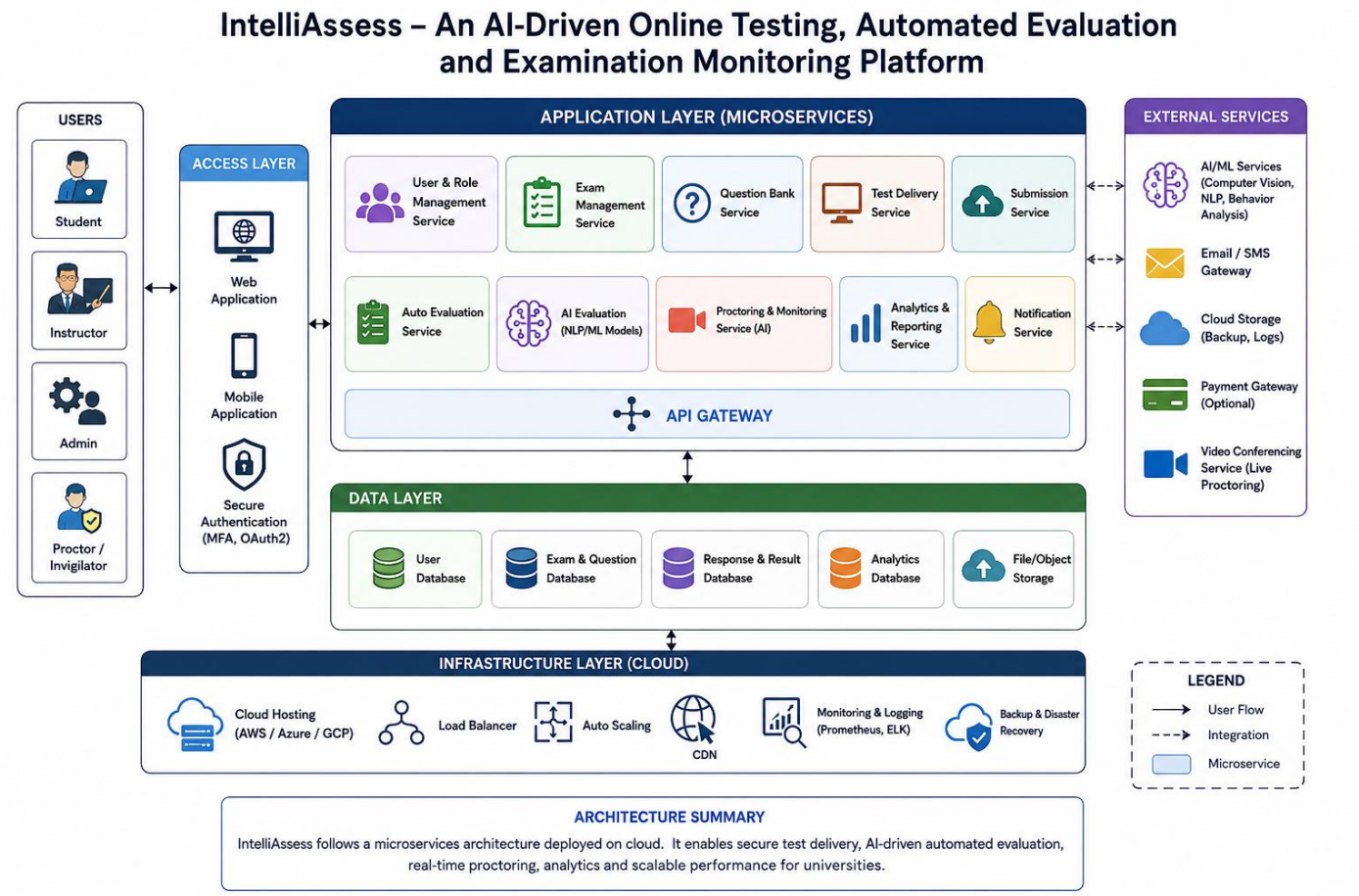
Style	Burst scalability	Availability / fault isolation	Independent evolution	Complexity / cost	Verdict
Single layered monolith (MVC)	**Poor - must scale everything**	**One failure takes all down**	**Low**	**Low**	Rejected: cannot isolate test delivery from reporting or AI load
Client-Server only (2-tier)	**Poor**	**Poor**	**Low**	**Low**	Rejected: insufficient for 5,000 concurrent users
Serverless (FaaS) only	**Excellent**	**Good**	**Good**	**Cold starts, execution time limits, hard to hold**	Partly used (workers) but not for timers / live sessions

				WebSocket state	
Classic SOA with ESB	**Medium**	**Medium (ESB bottleneck)**	**Medium**	**Heavy middleware**	Rejected: central bus becomes a bottleneck
Pure event-driven only	**Excellent**	**Good**	**Good**	**Hard to reason about request-response flows (login, paper load)**	Used only where asynchrony fits
Hybrid: Layered + Microservices + Event-Driven	**Excellent - scale only hot services**	**Excellent - failure isolation, multi-AZ**	**Excellent**	**Moderate (mitigated by managed Kubernetes / Kafka)**	**Selected**

2.3 Why the selected architecture is the most suitable

- **Peaks hit specific parts only.** Test Delivery and the gateway face 5,000 simultaneous users while Reporting is idle, and AI workers peak only after submission. Microservices let us auto-scale just those services instead of cloning an entire monolith.
- **Fault isolation protects the test.** If the AI proctor analyzer, AI answer scorer or report generator crashes, candidates keep writing the test; events wait in Kafka and are processed on recovery.
- **Asynchronous workloads fit event-driven design.** Image analysis, NLP scoring, grading and PDF generation need not block the candidate; they scale with queue length and can be replayed.
- **Layering keeps responsibilities clear** (presentation, edge, service, data), making security controls and testing simple.
- **Different data needs:** relational data (exams, marks), fast ephemeral state (timers, sessions), large binaries (snapshots) and immutable logs each get the store best suited to them.
- **Team and technology freedom:** the AI proctoring and evaluation services can use Python / ML libraries while core services use Java or Node.js, without affecting each other.

3. Draw the Software Architecture Diagram



3.1 Notation and reading guide

- **Tiers (dashed bands):** Actors, Presentation, Edge/Access, Service, Event-Driven Processing and Data tiers; external systems are shown in the right-hand column.
- **Boxes:** blue = business services / UI, yellow = security and edge components, purple = event broker and asynchronous workers, green = data stores, red = audit / immutable components, white = external systems.
- **Arrows:** solid double-headed = synchronous request-response (HTTPS / REST / gRPC / WSS / SQL); purple = asynchronous domain events on Kafka; dashed = external integrations (OIDC, REST, SMTP, replication, telemetry). Labels on the arrows state the protocol or data carried.

4. Explain Components and Their Interactions

4.1 Purpose and responsibilities of each component

Component	Responsibility	Technology (proposed)
Candidate Test Client (PWA)	Renders questions, countdown timer, secure mode (fullscreen lock, tab-switch / copy-paste detection), captures webcam snapshots and browser events	React, Service Worker, WebRTC / MediaDevices API
Local Autosave Buffer	Stores every answer locally and re-sends with exponential back-off so a network drop never loses work	IndexedDB, Background Sync
Faculty / Admin Portal	Paper setting, scheduling, proctor console, AI-score review, result moderation, dashboards	React SPA
CDN + WAF + API Gateway + Load Balancer	Serves static assets, filters attacks, terminates TLS, validates JWT, applies rate limits and routes requests	CloudFront / NGINX, Kong, AWS WAF
WebSocket Gateway	Keeps long-lived channels for heartbeats, proctor events and live alerts	Socket.IO / WSS behind L7 load balancer

Identity Provider + Identity & Access Service	SSO with university directory, MFA for staff, issues short-lived tokens, enforces RBAC	Keycloak (OIDC / SAML), JWT
Test Management Service	Question bank with AI-assisted question generation, paper templates, randomisation, scheduling, encrypted time-locked release	Spring Boot, PostgreSQL
Test Delivery Service	Starts sessions, serves the paper, keeps authoritative timer, saves answers idempotently, handles final submit and resume	Node.js / Spring Boot, Redis, PostgreSQL
AI Proctoring Service	Ingests events and snapshots, runs face / gaze / multi-person detection models, stores evidence, aggregates risk score, raises live alerts to proctors	Python FastAPI, OpenCV / MediaPipe, Redis, Object Storage
AI Evaluation & Results Service	Auto-grades objective answers, scores descriptive answers with NLP / LLM against rubrics, flags low-confidence answers for examiners, handles moderation and result publication	Spring Boot, Python (NLP / LLM), PostgreSQL
Notification / Reporting / ERP Adapter	Send messages; produce item-analysis and dashboards; translate between IntelliAssess and the ERP / SIS data models	REST adapters, SMTP / SMS APIs
Audit Service & Audit Store	Receives every critical event and writes it to an append-only, hash-chained store for compliance and disputes	WORM object store / append-only table
Config & Registry	Service discovery, feature flags, per-exam configuration	Kubernetes DNS / Consul
Event Broker + Workers	Decouples producers and consumers; Auto-Grading, AI Answer Scorer, AI Proctor Analyzer, Report Generator and Notification workers scale with queue lag	Apache Kafka, containerised consumers
Data stores	PostgreSQL (relational, primary + replica), Redis (sessions, timers), Object Storage (media, snapshots, PDFs), Analytics Warehouse (read model), Audit Store (immutable)	PostgreSQL, Redis, S3 / MinIO

4.2 How components communicate and exchange data

From - To	Mechanism	Data exchanged
Browser - Gateway	HTTPS / REST (JSON), TLS 1.3	Login, paper download, answer-save and submit calls with JWT
Browser - WebSocket Gateway	WSS	Heartbeats, tab-switch events, snapshot upload tokens, live proctor alerts
Gateway - Services	REST / gRPC, service discovery	Authenticated requests with user claims
Services - Identity Provider	OIDC / JWT validation	Tokens, roles, public keys (JWKS)
Services - Event Broker	Kafka (publish / subscribe)	Domain events: answer.saved, test.submitted, proctor.flag, ai.score.ready, result.published, notify.request
Workers - Data stores	SQL, cache protocol, S3 API	Scores, risk flags, snapshots, generated PDFs
ERP Adapter - University ERP / SIS	REST over HTTPS (OAuth2 client credentials)	Student and course master data in; final marks out
Notification Worker - Gateways	SMTP / REST	Hall tickets, reminders, result alerts
All services - Audit Service	Async events	Who did what, when, from which IP; hash of previous record

4.3 Overall workflow of the system

Setup & schedule -> Login (SSO + MFA) -> Paper release at start time -> Write test with autosave + AI proctoring -> Submit -> Auto-grade + AI scoring -> Examiner review & approval -> Result publish, ERP sync, notification

1. **Setup:** The Exam Cell syncs student and course data from the ERP through the adapter; faculty create questions (with AI-assisted generation) and papers; the schedule is saved and the encrypted paper is locked until the start time.
2. **Login:** The student opens the PWA from the CDN, is redirected to the institution SSO, receives a JWT; the gateway validates it on every call.
3. **Start:** At the scheduled time Test Delivery decrypts and serves the paper, creates a session record and starts a server-side timer stored in Redis.
4. **Write:** Each answer is saved locally in the browser and sent to Test Delivery every 10 s (idempotent, versioned). Simultaneously the client streams heartbeats and browser events over WSS and uploads webcam snapshots to Object Storage.
5. **Monitor:** Proctoring publishes proctor.flag events; the AI Proctor Analyzer scores each snapshot (face, gaze, multiple persons); the proctor console receives live alerts for high-risk candidates.
6. **Submit:** On submit (or timer expiry) Test Delivery freezes the session and publishes test.submitted.
7. **Evaluate:** The Auto-Grading Worker scores objective answers within seconds; the AI Answer Scorer grades descriptive answers against rubrics and routes low-confidence ones to examiners; moderation and approval follow.
8. **Publish:** On approval, result.published triggers the Report Generator (mark sheets), the ERP adapter (marks sync), the Notification worker (alerts) and the Analytics warehouse update.
9. **Audit:** Every step above also writes an immutable audit record that auditors can inspect at any time.

5. Discuss Quality Attributes

Attribute	How the architecture addresses it (tactics)	Target / evidence
Scalability	Stateless services behind a load balancer scale horizontally (Kubernetes HPA on CPU and request rate); Kafka partitions let consumer and AI workers scale with queue lag; Redis absorbs hot reads/writes; CDN offloads static content; scheduled pre-scaling before known test windows; read replicas and a separate analytics store keep reporting away from the test database.	5,000 concurrent users now; 50,000 by adding replicas and partitions, no redesign
Security	SSO + MFA for staff; JWT with short expiry; RBAC at gateway and service level; TLS 1.3 in transit, AES-256 at rest; papers encrypted and time-locked; WAF, rate limiting and DDoS protection; secrets in a vault; webcam data access limited to proctors / controller and auto-deleted after retention period; hash-chained audit log of AI scores and overrides; per-service least-privilege DB accounts.	OWASP Top 10 controls; no paper visible before start time; tamper-evident logs
Performance	Redis cache for sessions, timers and latest answers; async persistence to PostgreSQL; CDN for assets; paper pre-fetch (encrypted) before start; adaptive snapshot size; non-blocking, autoscaled workers for AI scoring, proctoring analysis, grading and PDFs; connection pooling and indexed queries.	Answer-save p95 < 300 ms; paper load < 3 s; objective results < 10 min; AI descriptive scoring < 30 min
Reliability	Client-side autosave with retry; idempotent APIs; server-authoritative timer with grace extension on verified outage; Kafka retention allows event replay; circuit breakers and retries between services; database replication, WAL archiving and nightly backups; graceful degradation (test continues even if AI proctoring, AI scoring or reporting is down).	Zero loss of acknowledged answers; RPO <= 5 min; RTO <= 30 min
Maintainability	Services aligned to business domains with small codebases; database-per-service and versioned REST / event contracts; versioned AI model registry; independent CI/CD pipelines and canary releases; configuration and feature flags externalised; centralised logs, metrics and tracing; contract and integration tests; infrastructure as code.	Change in one service deployable without touching others; fast fault diagnosis

Availability	Multi-AZ deployment, at least two replicas of each service, health checks with automatic restart, load balancer failover, managed PostgreSQL with automatic failover, replicated Kafka and Redis, DR region with backups, blue-green deployment, freeze window during exams, 24x7 monitoring and alerting.	99.9% in exam windows; no single point of failure

6. Justify Architectural Decisions

6.1 Rationale behind the architectural choices

ID	Decision	Rationale	Alternative considered
D1	Domain-based microservices instead of a monolith	Independent scaling of Test Delivery / AI Proctoring / AI Evaluation and isolation of failures during peak load	Modular monolith (simpler, but cannot scale or fail independently)
D2	Event broker (Kafka) for grading, AI scoring, AI analysis, reports and notifications	Decouples slow tasks from the candidate path; durable and replayable events; smooths bursts	Direct synchronous calls (cascading failures); simple queue (no replay)
D3	Offline-tolerant client with local autosave and idempotent save API	Candidate networks are unreliable; guarantees no answer loss	Server-only saving (data loss on disconnect)
D4	Database-per-service with polyglot storage (PostgreSQL, Redis, S3, WORM)	Each data type gets the right consistency, speed and cost; loose coupling between teams	One shared database (tight coupling, contention)
D5	API Gateway + external Identity Provider (OIDC)	Central security enforcement and reuse of the university SSO; no password storage	Custom authentication in every service
D6	Separate immutable Audit Service	Supports disputes, re-evaluation and NAAC / NBA evidence; records every AI score and human override; separation of duty	Application logs in the main DB (editable)
D7	ERP integration through an adapter (anti-corruption layer)	Isolates IntelliAssess from ERP model changes; one place to change if the ERP is replaced	Direct database access to ERP (fragile, insecure)
D8	Containers on Kubernetes across multiple AZs	Auto-scaling, self-healing, rolling updates, portable across cloud / on-premise	Fixed VMs (over-provisioned, manual scaling)

6.2 Trade-offs considered

Trade-off	Cost accepted	Mitigation
Microservices vs. simplicity	Higher operational complexity, network latency, distributed debugging	Limit to about 8 coarse-grained services; managed Kubernetes / Kafka; distributed tracing; shared libraries and templates
Eventual vs. strong consistency	Grading, AI scores, reports and dashboards lag the raw answers by seconds	Strong consistency only for answer save and submit; eventual consistency for derived data with clear UI status
Security vs. usability / privacy	Webcam monitoring and secure mode may feel intrusive and strain low-end devices	Consent and privacy notice, human review of AI flags (AI never auto-fails a student), adaptive quality, retention limits
Cost vs. availability	Multi-AZ replicas and DR increase cost	Auto-scale down outside exam windows; reserve capacity only for scheduled peaks
Client autonomy vs. integrity	Local buffering means the client holds answers briefly	Server-authoritative timer, signed and versioned saves, hash verification at submit

6.3 Support for future enhancements, integration and long-term maintainability

- **New features as new services or consumers:** plagiarism detection for descriptive answers, adaptive testing, coding-exam sandboxes, voice proctoring or AI feedback for students can subscribe to existing Kafka topics without modifying current services (open-closed principle).

- **Replaceable AI models:** the AI Proctor Analyzer and AI Answer Scorer are workers behind stable event contracts, so face / gaze models and NLP / LLM scorers can be upgraded or A/B tested independently.
- **Integration-ready:** versioned REST APIs and OIDC / SAML allow LMS (Moodle), digilocker, accreditation portals or other campuses to be integrated through adapters.
- **Multi-tenant growth:** tenant identifiers in tokens and data partitioning allow several colleges to share one deployment.
- **Evolution path:** services are independently versioned; a rarely used service can be merged or a hot one split further; Kubernetes and containers avoid cloud vendor lock-in.
- **Operational maintainability:** automated CI/CD, infrastructure as code, standard logging / metrics, runbooks and architecture decision records (ADRs) keep the system understandable as the team changes.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	**/25**